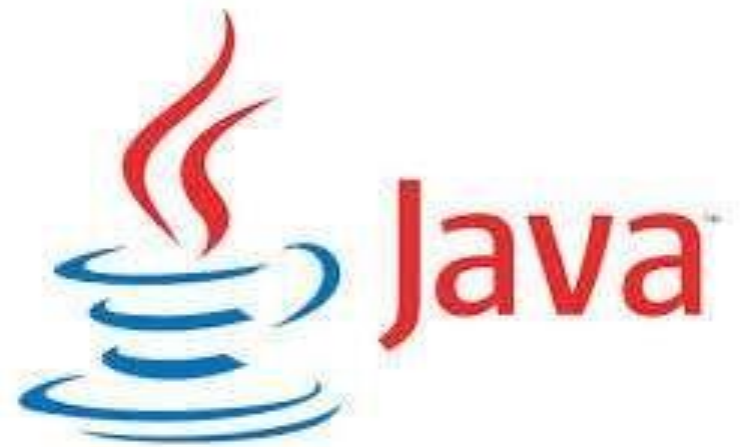




SOC2030

Application Programming in Java



Week 10 GUI FX

Dr. Andrei Dragunov

Intro.

<https://docs.oracle.com/javase/8/javase-clienttechnologies.htm> (!!!)

Most Java textbooks that introduce GUI programming provide hand-coded GUIs—that is, the authors build the GUIs from scratch in Java code, rather than using a visual GUI design tool. This is due to the fractured Java IDE market—there are many Java IDEs, so authors can't depend on any one IDE being used, and each generates different code.

JavaFX is organized differently. The Scene Builder tool is a standalone JavaFX GUI visual layout tool that can also be used with various IDEs, including the most popular ones—Eclipse, IntelliJ IDEA and NetBeans. You can download **Scene Builder** at:

<https://gluonhq.com/products/scene-builder/>

JavaFX Scene Builder

JavaFX Scene Builder enables you to create GUIs by dragging and dropping GUI components from Scene Builder's library onto a design area.

You can use **Cascading Style Sheets (CSS)** to change the entire look-and-feel of your GUI—a concept sometimes called **skinning**.

As you create and modify a GUI, JavaFX Scene Builder generates **FXML (FX Markup Language)**—an XML vocabulary for defining and arranging JavaFX GUI controls without writing any Java code.

XML (eXtensible Markup Language) is a widely used language for describing things— it's readable both by computers and by humans. In JavaFX, FXML concisely describes GUI, graphics and multimedia elements. **You do not need to know FXML or XML to study Java FX.**

FXML code

The FXML code is separate from the program logic that's defined in Java source code—this separation of the interface (the GUI) from the implementation (the Java code) makes it easier to debug, modify and maintain JavaFX GUI apps.

JavaFX Application

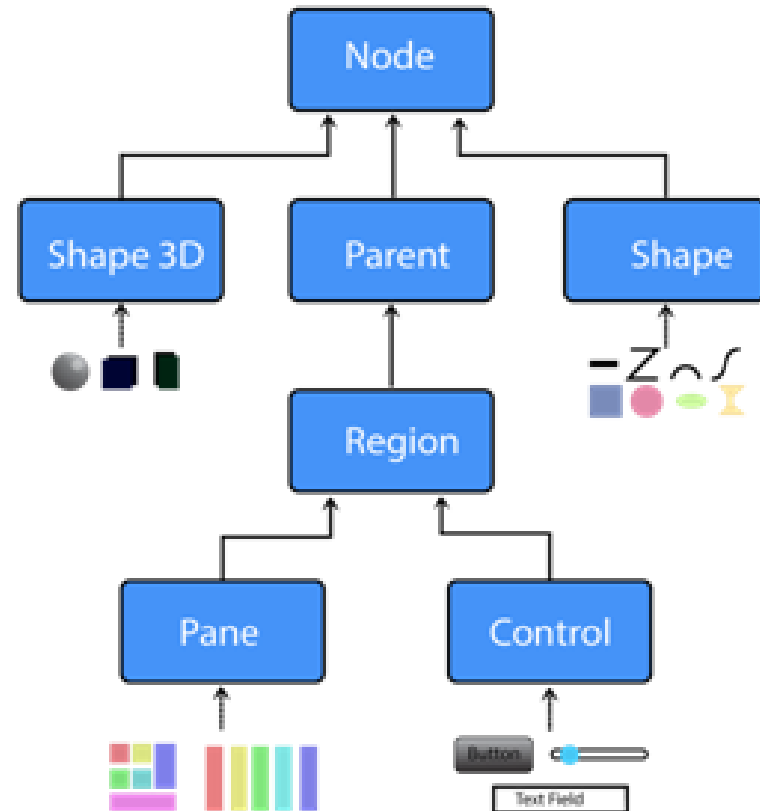
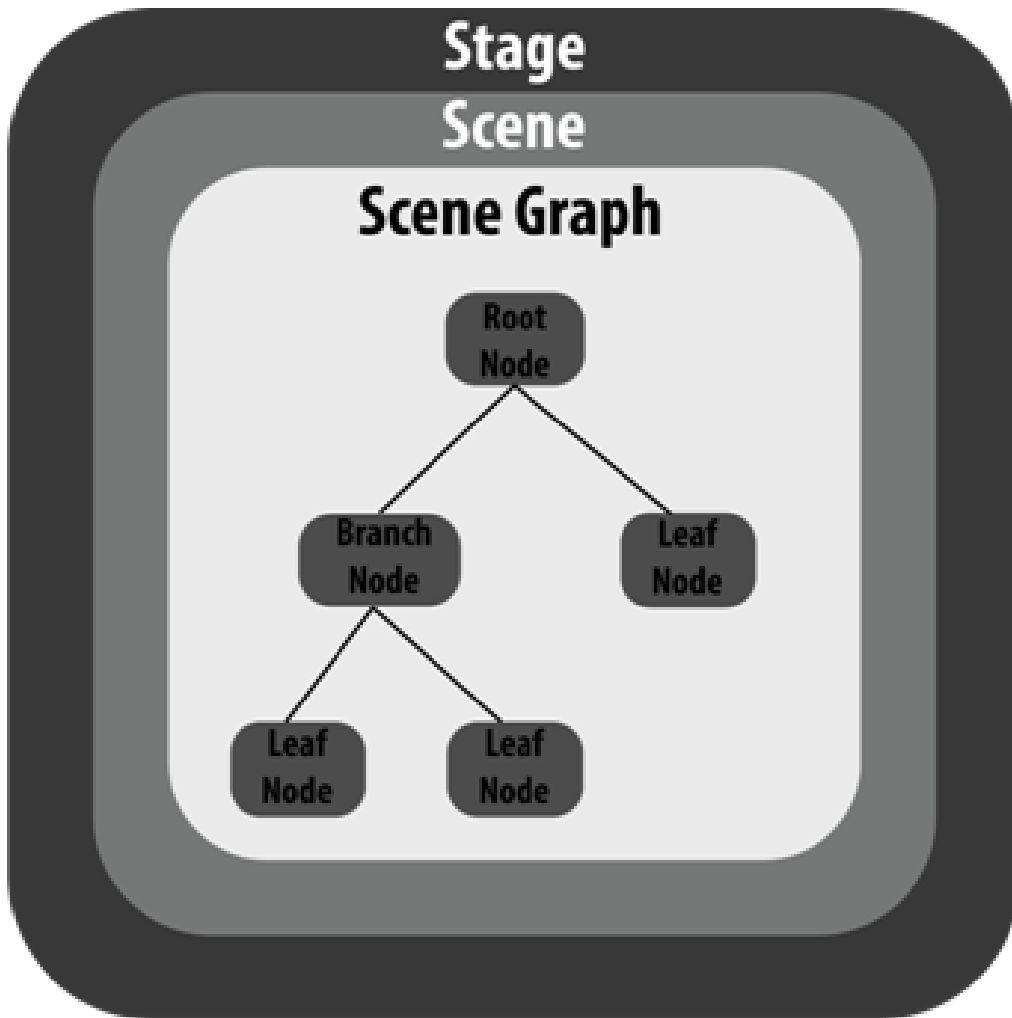
JavaFX application is divided hierarchically into three main components known as **Stage**, **Scene** and **nodes**. We need to import `javafx.application.Application` class in every JavaFX application. This provides the following life cycle methods for JavaFX application.

1. `public void init()`
2. `public abstract void start(Stage primaryStage)`
3. `public void stop()`

in order to create a basic JavaFX application, we need to:

1. Import `javafx.application.Application` into our code.
2. Inherit `Application` into our class.
3. Override `start()` method of `Application` class.

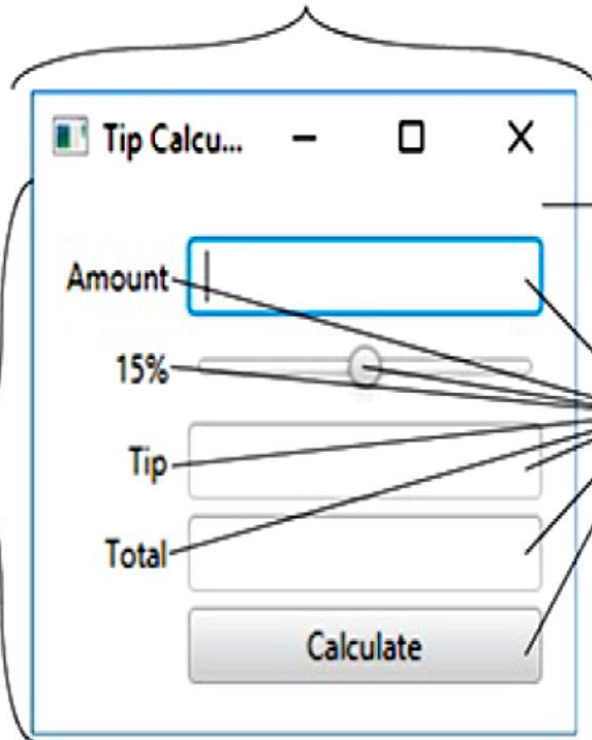
JavaFX Application Structure



JavaFX Application Structure

The window is known as the stage

The stage contains a scene graph of nodes



The root node of this scene graph is a layout container that arranges the other nodes

Each of the JavaFX components in this GUI is a node in the scene graph

Stage

- Stage in a JavaFX application is similar to the Frame in a Swing Application.
- It acts like a container for all the JavaFX objects.
- Primary Stage is created internally by the platform.
- Other stages can further be created by the application.
- The object of primary stage is passed to start method.
- We need to call show method on the primary stage object in order to show our primary stage.

Scene

Scene actually holds all the physical contents (nodes) of a JavaFX application.

`javafx.scene.Scene` class provides all the methods to deal with a scene object. Creating scene is necessary in order to visualize the contents on the stage.

At one instance, the scene object can only be added to one stage. In order to implement Scene in our JavaFX application, we must import `javafx.scene` package in our code. The Scene can be created by creating the Scene class object and passing the layout object into the Scene class constructor.

Scene Graph

Scene Graph exists at the lowest level of the hierarchy. It can be seen as the collection of various nodes. A node is the element which is visualized on the stage. **It can be any button, text box, layout, image, radio button, check box, etc.**

The nodes are implemented in a tree kind of structure. There is always one root in the scene graph. This will act as a parent node for all the other nodes present in the scene graph. However, this node may be any of the layouts available in the JavaFX system.

The leaf nodes exist at the lowest level in the tree hierarchy. Each of the node present in the scene graphs represents classes of javafx.scene package therefore we need to import the package into our application in order to create a full featured javafx application.

First JavaFX Application

Step 1: Extend class Application and
override start()

Step 2: Create a Button

Step 3: Create a layout and add button to it

Step 4: Create a Scene

Step 5: Prepare the Stage

Step 6: Create an event for the button

Step 7: Create the main method

Enjoy! ;)

```

public class Test extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
        Button btn1=new Button("Say, Hello World");
        btn1.setOnAction(new EventHandler<ActionEvent>() {

            @Override
            public void handle(ActionEvent arg0) {
                // TODO Auto-generated method stub
                System.out.println("hello world");
            }
        });
        StackPane root=new StackPane();
        root.getChildren().add(btn1);
        Scene scene=new Scene(root,600,400);
        primaryStage.setTitle("First JavaFX Application");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main (String[] args)
    {
        launch(args);
    }
}

```

Using Built-in Layout Panes

A JavaFX application can manually lay out the UI by setting the position and size properties for each UI element. However, an easier option is to make use of layout panes. The JavaFX SDK provides several layout panes for the easy setup and management of classic layouts such as rows, columns, stacks, tiles, and others. As a window is resized, the layout pane automatically repositions and resizes the nodes that it contains according to the properties for the nodes.

BorderPane

HBox

VBox

StackPane

GridPane

FlowPane

TilePane

AnchorPane

Styling UI Controls with CSS

Style sheets contain style definitions that control the look of user interface elements. Using CSS in JavaFX applications is similar to using CSS in HTML. JavaFX CSS are based on the W3C CSS version 2.1 specification (available at <http://www.w3.org/TR/CSS21/>) with some additions from current work on version 3 of the specification and some extensions that support specific JavaFX features.

Default Style Sheet

The default style sheet for JavaFX applications is `modena.css`, which is found in the JavaFX runtime JAR file, `jfxrt.jar`. This style sheet defines styles for the root node and the UI controls. To view this file, go to the `\jre\lib\ext` directory under the directory in which the Java Development Kit (JDK) is installed. Use the following command to extract the style sheet from the JAR file:

```
jar xf jfxrt.jar  
com/sun/javafx/scene/control/skin/modena/modena.css
```

Creating Style Sheets

You can create one or more of your own style sheets to override the styles in the default style sheet and to add your own styles. Typically style sheets that you create have an extension of .css and are located in the same directory as the main class for your JavaFX application.

Adding a Style Sheet

```
String css =  
Test.class.getResource("testCSS.css").toExternalForm();  
scene.getStylesheets().add(css);
```


JavaFX - Event Handling

In JavaFX, we can develop GUI applications, web applications and graphical applications. In such applications, whenever a user interacts with the application (nodes), an event is said to have been occurred.

For example, clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from list, scrolling the page are the activities that causes an event to happen.

Types of Events

The events can be broadly classified into the following two categories –

- **Foreground Events** – Those events which require the direct interaction of a user. They are generated as consequences of a person interacting with the graphical components in a Graphical User Interface. For example, clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from list, scrolling the page, etc.
- **Background Events** – Those events that don't require the interaction of end-user are known as background events. The operating system interruptions, hardware or software failure, timer expiry, operation completion are the example of background events.

Events in JavaFX

JavaFX provides support to handle a wide varieties of events. The class named Event of the package javafx.event is the base class for an event.

An instance of any of its subclass is an event. JavaFX provides a wide variety of events. Some of them are are listed below.

Mouse Event – This is an input event that occurs when a mouse is clicked. It is represented by the class named **MouseEvent**. It includes actions like mouse clicked, mouse pressed, mouse released, mouse moved, mouse entered target, mouse exited target, etc.

Key Event – This is an input event that indicates the key stroke occurred on a node. It is represented by the class named **KeyEvent**. This event includes actions like key pressed, key released and key typed.

Drag Event – This is an input event which occurs when the mouse is dragged. It is represented by the class named **DragEvent**. It includes actions like drag entered, drag dropped, drag entered target, drag exited target, drag over, etc.

Window Event – This is an event related to window showing/hiding actions. It is represented by the class named **WindowEvent**. It includes actions like window hiding, window shown, window hidden, window showing, etc.

Event Handling

Event Handling is the mechanism that controls the event and decides what should happen, if an event occurs. This mechanism has the code which is known as an event handler that is executed when an event occurs.

JavaFX provides handlers and filters to handle events. In JavaFX every event has –

Target – The node on which an event occurred. A target can be a window, scene, and a node.

Source – The source from which the event is generated will be the source of the event. In the above scenario, mouse is the source of the event.

Type – Type of the occurred event; in case of mouse event – mouse pressed, mouse released are the type of events.

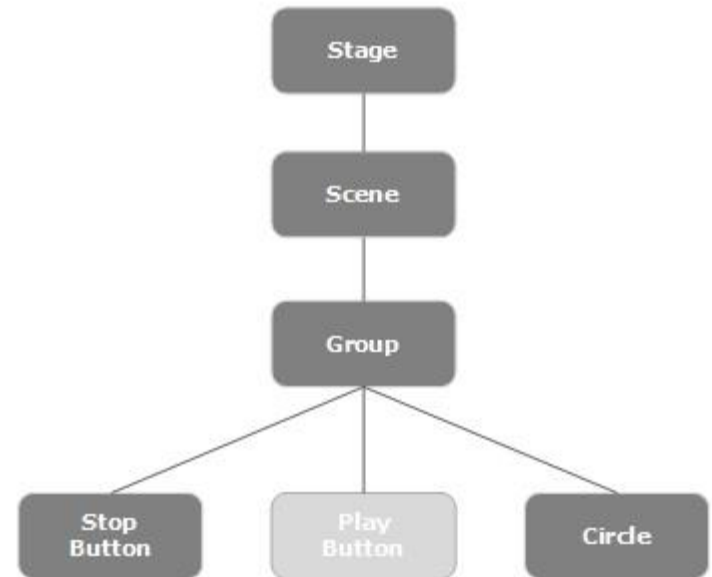
Phases of Event Handling in JavaFX

Whenever an event is generated, JavaFX undergoes the following phases.

Route Construction

Whenever an event is generated, the default/initial route of the event is determined by construction of an Event Dispatch chain. It is the path from the stage to the source Node.

Following is the event dispatch chain for the event generated, when we click on the play button in the above scenario.



Event Capturing Phase

After the construction of the event dispatch chain, the root node of the application dispatches the event. This event travels to all nodes in the dispatch chain (from top to bottom). If any of these nodes has a filter registered for the generated event, it will be executed. If none of the nodes in the dispatch chain has a filter for the event generated, then it is passed to the target node and finally the target node processes the event.

Event Bubbling Phase

In the event bubbling phase, the event is travelled from the target node to the stage node (bottom to top). If any of the nodes in the event dispatch chain has a handler registered for the generated event, it will be executed. If none of these nodes have handlers to handle the event, then the event reaches the root node and finally the process will be completed.

Event Handlers and Filters

Event filters and handlers are those which contains application logic to process an event. A node can register to more than one handler/filter. In case of parent–child nodes, you can provide a common filter/handler to the parents, which is processed as default for all the child nodes.

As mentioned above, during the event, processing is a filter that is executed and during the event bubbling phase, a handler is executed. All the handlers and filters implement the interface `EventHandler` of the package `javafx.event`.

Adding and Removing Event Filter

```
//Creating the mouse event handler
EventHandler<MouseEvent> eventHandler = new
EventHandler<MouseEvent>() {
    @Override
    public void handle(MouseEvent e) {
        System.out.println("Hello World");
        circle.setFill(Color.DARKSLATEBLUE);
    }
};

//Adding event Filter
Circle.addEventFilter(MouseEvent.MOUSE_CLICKED,
eventHandler);
```